

Memory as Dynamics: A Unified Model of Distributed Sequence Encoding in Recurrent Neural Systems

Attractor memories, trajectory-coded sequences, and controlled recurrent dynamics in a single substrate

Vyom Paknikar

Independent researcher

hello@vyompaknikar.com

Document type
Manuscript draft, v1.0

Date
21 September 2026

Target venues
Neuro-AI and neuromorphic
journals

Format
A4 portrait, single column

Abstract

Biological neural systems exhibit remarkable memory capacity and energy efficiency by embedding memory within distributed neural dynamics rather than in explicit storage modules. Inspired by this principle, we propose a unified computational framework in which memories are represented as stable attractor states and sequences as trajectories within recurrent neural networks. Our model integrates dendritic computation, Hebbian-temporal learning, and controlled recurrent dynamics to enable content-addressable recall and sequence reconstruction. We present a formal mathematical formulation and simulation framework demonstrating robust memory storage, resistance to noise, and scalable sequence encoding. This approach offers theoretical insights into biological cognition and practical implications for neuromorphic and energy-efficient artificial intelligence systems.

Keywords: attractor networks; sequence memory; recurrent neural networks; Hebbian learning; spike-timing-dependent plasticity; dendritic computation; neuromorphic computing; energy-efficient AI

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Problem statement and aims	3
2	Related work	4
3	Theoretical framework	6
3.1	Memory as attractor states	6
3.2	Sequence encoding	6
3.3	Control and stability	7
3.4	Dendritic subunits	7
4	Mathematical formulation	9
4.1	Network state dynamics	9
4.2	Attractor stability	9
4.3	Learning rules	10
4.4	Sequence encoding	10
4.5	Gated dynamics	10
4.6	Dendritic extension	11
4.7	Capacity analysis	11
4.8	Energy model	11
5	Methods and simulation framework	13
5.1	Environment	13
5.2	Network architecture and parameters	13
5.3	Training procedure	13
5.4	Evaluation tasks and metrics	14
5.5	Baselines	14
5.6	Reference implementation	14
6	Results: targets and protocol	16
7	Discussion	17
8	Limitations	17
9	Future work	18
10	Conclusion	18
	References	19
A	Derivation notes	24
A.1	Jacobian and the role of saturation	24
A.2	Crosstalk in sequence retrieval	24
B	Revision notes for the authors	25

1 Introduction

1.1 Motivation

Contemporary artificial intelligence largely separates computation from memory. Transformers retrieve from an explicit key-value context (Vaswani et al. 2017), memory-augmented networks read and write an external matrix (Graves et al. 2014), and large models keep knowledge in parameterized embeddings. This mirrors the von Neumann architecture, whose shared path between processor and memory is a long-recognized bottleneck (Backus 1978) and a dominant energy cost in present-day hardware (Horowitz 2014; Sebastian et al. 2020).

Biological brains take a different route. Signaling in grey matter is expensive enough that only a small fraction of neurons can be substantially active at any moment (Attwell & Laughlin 2001; Lennie 2003), so memory and computation must share the same sparse, low-rate substrate. The neuroscientific evidence points to memory being embedded in the dynamics of recurrent circuits: persistent activity during working memory (Goldman-Rakic 1995; Wang 2001), attractor dynamics in oculomotor and navigation circuits (Seung 1996; Khona & Fiete 2022), and temporally structured replay of hippocampal firing sequences (Skaggs & McNaughton 1996; Buzsáki 2010; Lisman & Jensen 2013). Population-level analyses increasingly describe cortical computation as trajectories through a neural state space (Mante et al. 2013; Vyas et al. 2020), and reviews of memory theory emphasize the same recurrent, distributed picture (Chaudhuri & Fiete 2016).

The separation between computation and memory in artificial systems limits scalability, adaptability, and energy efficiency. This work proposes a unified architecture in which memory, computation, and sequence processing emerge from intrinsic network dynamics.

1.2 Problem statement and aims

We ask whether a single recurrent substrate, without a separate memory module, can (i) store discrete items as stable states, (ii) store ordered sequences as trajectories through those states, (iii) retrieve both from partial or noisy cues, and (iv) do so at a bounded, analyzable energy cost. Existing models address subsets of these requirements (Section 2). We aim to:

1. **Formalize memory as attractor dynamics.** Define a memory as a stable region of state space and give conditions for its existence and stability (Sections 3.1, 4.2).
2. **Model sequences as state-space trajectories.** Encode order through temporally asymmetric plasticity and control transitions with a gating subsystem (Sections 3.2–3.3, 4.4–4.5).
3. **Demonstrate computational viability.** Specify a reproducible simulation study covering pattern recall, sequence recall, and interference (Section 5).
4. **Analyze scaling and efficiency.** Derive capacity estimates and an event-based energy model, and state how each will be tested (Sections 4.7–4.8, 6).

Section 2 positions the work against prior models. Sections 3 and 4 develop the theory; Section 5 specifies the experiments; Section 6 states the targeted outcomes; Sections 7–10 discuss implications, limits, and extensions. Appendices give derivation notes and a list of revisions made while preparing this document from the original draft.

2 Related work

Table 1 summarizes the six families most often cited as models of dynamic memory (Hopfield networks, reservoir computing, LSTMs, predictive coding, spiking networks and continuous attractors) together with three further families that bear directly on sequences and long-range state. Each implements a partial form of dynamic memory.

Table 1. Prior models of dynamic memory, the mechanism each uses, and the gap this work addresses.

Model family	How memory is held	Sequence support	Main limitation for our aims	Key sources
Hopfield-type associative networks	Fixed-point attractors of a symmetric weight matrix, with an energy function	None natively; asymmetric or delayed couplings add transitions	Capacity of about $0.14N$ for Hebbian storage; spurious states; no control of transitions	(Hopfield 1982; Amit et al. 1985; Sompolinsky & Kanter 1986; Kleinfeld 1986)
Modern (dense) Hopfield networks	Higher-order energies; very large capacity	Mostly static retrieval	Close to attention; explicit pattern storage	(Krotov & Hopfield 2016; Demircigil et al. 2017; Ramsauer et al. 2021)
Reservoir computing and echo state networks	Fading echoes of input in a fixed random recurrent core; trained readout	Yes, implicit and input-driven	Memory fades by design; no persistent stable states	(Jaeger 2001; Maass et al. 2002; Jaeger & Haas 2004; Lukoševičius & Jaeger 2009; Ganguli et al. 2008)
LSTM and gated RNNs	Learned gated cell state	Yes, learned by backpropagation through time	Explicit memory cell; gradient-based training; dense arithmetic	(Hochreiter & Schmidhuber 1997; Bengio et al. 1994; Pascanu et al. 2013)
Predictive coding	Generative model and prediction errors	Via hierarchical temporal prediction	An inference framework rather than a storage mechanism	(Rao & Ballard 1999; Friston 2010)
Spiking neural networks	Spike timing, sparse events, local plasticity	Yes, through timing codes	Training difficulty; device variability	(Maass 1997; Gerstner et al. 2014; Bellec et al. 2020; Neftci et al. 2019)
Continuous attractors	Manifolds of neutral stability for analog variables	Limited; drift along the manifold	Fine-tuning; not discrete item memory	(Amari 1977; Ben-Yishai et al. 1995; Seung 1996; Khona & Fiete 2022)
Transient and heteroclinic dynamics	Sequences as saddle-to-saddle trajectories	Yes	Noise sensitivity; learning rules underdeveloped	(Rabinovich et al. 2001; Rabinovich et al. 2008)

Table 1 (continued)

Model family	How memory is held	Sequence support	Main limitation for our aims	Key sources
Attention and structured state-space models	Explicit key–value context, or a learned linear recurrent state	Yes	Not biologically grounded; explicit memory or dense compute	(Vaswani et al. 2017; Gu et al. 2022; Gu & Dao 2023)

Associative and attractor models. Hopfield (1982) showed that a symmetric recurrent network with Hebbian weights has an energy function whose minima are stored patterns, giving content-addressable recall; Amit et al. (1985) established its capacity limit of roughly 0.14 patterns per neuron for random binary patterns. Asymmetric and delayed couplings turn such networks into sequence generators (Sompolinsky & Kanter 1986; Kleinfeld 1986; Amit 1988), but the resulting dynamics lack a Lyapunov function and offer no principled way to hold, advance, or halt a sequence. Dense associative memories raise capacity substantially (Krotov & Hopfield 2016; Demircigil et al. 2017) and connect to attention (Ramsauer et al. 2021), at the price of explicit pattern storage.

Reservoir and gated recurrent networks. Echo state and liquid state machines (Jaeger 2001; Maass et al. 2002) exploit a fixed random recurrent core whose short-term memory is a fading trace of past input (Ganguli et al. 2008); learning is confined to a readout, which is efficient but leaves no persistent attractors. Operating near the edge of chaos maximizes the trace length (Bertschinger & Natschläger 2004), and feedback training of chaotic networks can generate complex trajectories (Sussillo & Abbott 2009). LSTMs (Hochreiter & Schmidhuber 1997) hold information in a gated cell and are trained by backpropagation through time, whose gradient pathologies are well documented (Bengio et al. 1994; Pascanu et al. 2013).

Predictive, spiking and continuous models. Predictive coding (Rao & Ballard 1999; Friston 2010) explains cortical responses through hierarchical prediction errors but is not, by itself, a mechanism for storing arbitrary items. Spiking networks (Maass 1997; Gerstner et al. 2014) supply the energy-relevant substrate, and recent training methods make them practical (Bellec et al. 2020; Neftci et al. 2019). Continuous attractor models (Amari 1977; Ben-Yishai et al. 1995; Seung 1996) hold analog variables on a manifold (Khona & Fiete 2022) but not discrete, addressable items. Transient-dynamics accounts encode sequences as trajectories between saddles (Rabinovich et al. 2001; Rabinovich et al. 2008).

Positioning. These approaches implement partial forms of dynamic memory but lack integrated sequence control and scalable stability. Our work extends them by combining three ingredients in one network: attractor stability from a symmetric, Hebbian component; ordered transitions from a temporally asymmetric component; and control dynamics that switch between the two.

3 Theoretical framework

3.1 Memory as attractor states

We define memory as a stable region in neural state space toward which activity converges under partial cues.

Definition 1 (attractor memory). A pattern ζ is stored if the network has a stable fixed point x^* close to ζ . Its *basin of attraction* is the set of initial states whose trajectories converge to x^* . *Content-addressable recall* means that any initial state inside the basin, such as a noisy or partial version of ζ , is completed to x^* .

The radius of the basin is a direct measure of robustness to noise, and the number of distinct stable fixed points bounds capacity. When the recurrent weights are symmetric and updates are asynchronous, the dynamics descend an energy function, so every trajectory ends in a fixed point (Hopfield 1982; Hopfield 1984); this guarantees convergence but not that the fixed points are the intended memories, which is why we track spurious states explicitly (Section 5.4). Figure 1 sketches the picture.

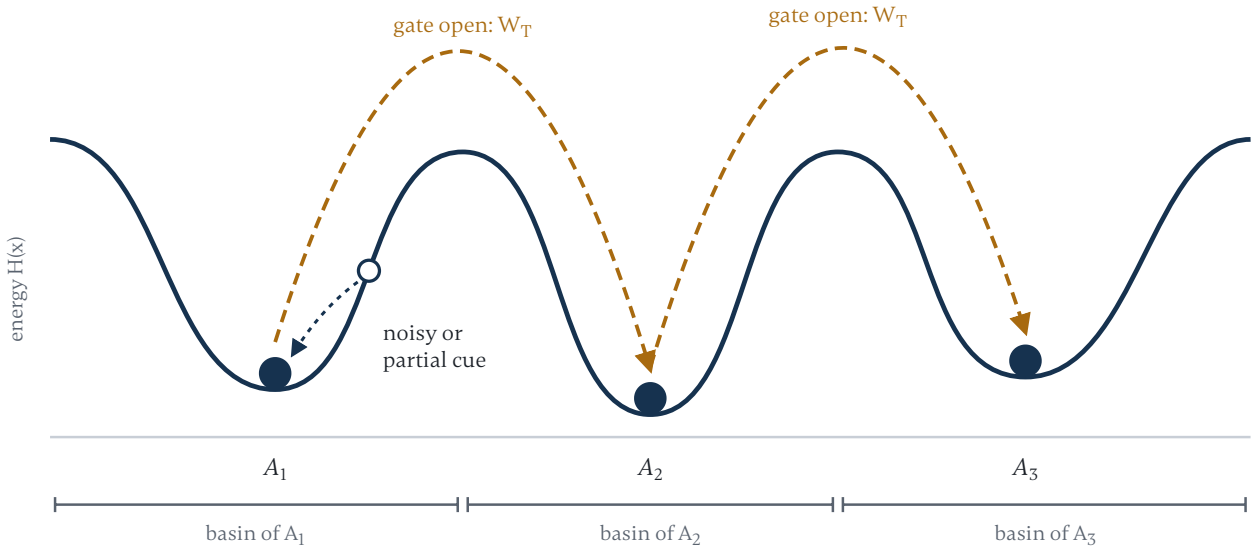


Figure 1. Schematic attractor landscape (not simulation output). Stored patterns A_1 – A_3 are wells of an energy function; a noisy cue inside a basin relaxes to its attractor (content-addressable recall). With the gate closed, states remain in their wells; when the gate opens, the asymmetric term W_T pushes the state into the next basin (dashed arcs), so a sequence is a trajectory through ordered attractors. The horizontal axis stands for the high-dimensional state space projected onto one dimension.

3.2 Sequence encoding

Sequences are encoded as ordered transitions between attractors. Learning associates successive states through temporally asymmetric plasticity: a synapse from a neuron active at time t onto a neuron active at time $t+1$ is strengthened, the network-level counterpart of spike-timing-dependent plasticity, in which presynaptic activity that precedes postsynaptic firing potentiates the synapse and the reverse order depresses it (Markram et al. 1997; Bi & Poo 1998; Song et al. 2000). In abstract networks, asymmetric couplings of this kind were shown to generate temporal association and sequential state generation (Sompolsky & Kanter 1986; Kleinfeld 1986).

A retrieved sequence is therefore a trajectory $x(t)$ that visits attractors $x_1, x_2, \dots, x_k$ in order and dwells near each. This differs from encodings in which order is carried by an external clock or by input alone, and from winnerless-competition schemes that build sequences from the connectivity of saddle points (Rabinovich et al. 2001; Rabinovich et al. 2008); here, order is written into synaptic weights that share a matrix with the item memories.

3.3 Control and stability

A gating subsystem regulates transitions and suppresses chaotic regimes, analogous to prefrontal and hippocampal modulation. Two separate problems require control.

Switching between holding and advancing. A symmetric component makes states stable but keeps a sequence stuck in its first attractor; an asymmetric component advances the sequence but removes the energy function. We therefore write the recurrent matrix as the sum of a symmetric attractor part W_A and a gated asymmetric transition part W_T . With the gate closed the network is a gradient system with stable memories; with the gate open the flow becomes non-gradient and states advance. The gate plays the role that prefrontal gating plays in models of working memory (Miller & Cohen 2001; O'Reilly & Frank 2006), and its rhythmic form parallels the theta-gamma coding of item order in the hippocampus (Lisman & Jensen 2013).

Avoiding chaos. Random recurrent networks with sufficiently large gain become chaotic (Sompolinsky et al. 1988), which would destroy retrieval. Input drive suppresses chaos (Rajan et al. 2010), and computation is often richest near, but not beyond, the boundary (Bertschinger & Natschl ger 2004). We keep the effective gain below the chaotic threshold outside retrieval by regularizing weights (Section 4.3) and monitor the largest Lyapunov exponent during simulations (Section 5.4).

3.4 Dendritic subunits

The abstract commits the model to dendritic computation. Pyramidal-neuron dendrites contain nonlinear integration sites (Larkum et al. 1999) that can be idealized as a two-layer network: sigmoidal branch subunits feeding a somatic sum (Poirazi et al. 2003). Active dendrites and structural plasticity increase the memory capacity attainable from a fixed synaptic budget (Poirazi & Mel 2001), and dendritic integration is a general computational resource in neurons (London & H usser 2005). We include a branch-nonlinearity extension of the point-neuron update (Eq. 13) and treat it as an optional module, evaluated by ablation against the point-neuron model.

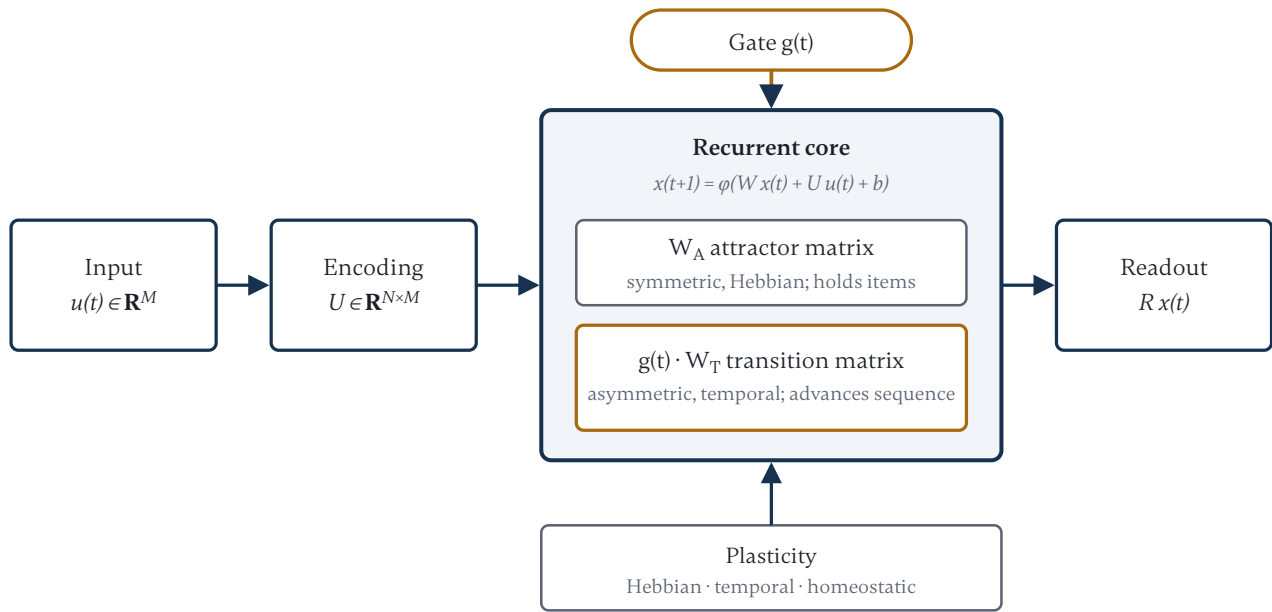


Figure 2. Architecture: Input $\rightarrow$ Encoding $\rightarrow$ Recurrent core $\rightarrow$ Readout. The recurrent core superposes a symmetric attractor matrix W_A (holds items) and a gated asymmetric matrix W_T (advances sequences). A gate $g(t)$ scales W_T ; plasticity rules update the recurrent weights during learning.

4 Mathematical formulation

This section is the formal core of the paper. Vectors are columns, matrices are upper-case, and W_{ij} is the weight from neuron j onto neuron i . Table 2 lists the symbols.

Table 2. Notation.

Symbol	Dimension	Meaning
$x(t)$	$\mathbf{R}^N$	Neural population activity at time step t
W	$\mathbf{R}^{N \times N}$	Recurrent weights; $W = W_A + g(t) W_T$ in the gated model
$U, u(t)$	$\mathbf{R}^{N \times M}, \mathbf{R}^M$	Input weights and input vector
b	$\mathbf{R}^N$	Bias (baseline drive)
φ	$\mathbf{R} \rightarrow \mathbf{R}$	Nonlinear activation, $\varphi(z) = \tanh(\beta z)$, gain $\beta > 0$ (a sigmoid is an alternative)
ξ^μ, x_i	$\mathbf{R}^N$	Stored pattern $\mu = 1 \dots P$; x_i is the i -th element of a sequence of length k
η, λ, γ	scalars	Hebbian, temporal, and homeostatic rates
$g(t)$	$[0, g_{\max}]$	Gate that scales the transition matrix
$m_\mu(t)$	$[-1, 1]$	Overlap between state and pattern μ , $m_\mu = \xi^\mu \cdot x / (\ \xi^\mu\ \ x\)$
$s_i(t), E_{\text{spike}}$	$\{0, 1\}$, joules	Spike indicator of neuron i ; energy per spike

4.1 Network state dynamics

The population evolves in discrete time under a recurrent nonlinear map with input:

$$x(t+1) = \varphi(W x(t) + U u(t) + b) \quad (1)$$

where φ is a saturating nonlinearity ($\tanh$ or sigmoid). Saturation keeps activity bounded and is what allows stable fixed points to coexist with an unstable quiescent state (Section 4.2).

4.2 Attractor stability

An autonomous memory state x^* ($u = 0$) is a fixed point of Eq. (1):

$$x^* = \varphi(W x^* + b) \quad (2)$$

It is locally asymptotically stable if the Jacobian of the map at x^* has spectral radius below one:

$$J(x^*) = \text{diag}(\varphi'(W x^* + b)) W \quad (3)$$

$$\rho(J(x^*)) < 1 \quad (4)$$

where ρ is the spectral radius. Two consequences guide the design. First, saturated units have $\varphi' \approx 0$, so a state with most units near ± 1 is strongly contracting; stored patterns are stable for this reason. Second, the quiescent state $x = 0$ has Jacobian βW (for $b = 0$), so memories can be retrieved only if $\rho(\beta W) > 1$ there, that is, if the origin is unstable and cues are amplified toward an attractor rather than decaying. A model in which W is scaled so that $\rho(\beta W) < 1$ everywhere has no non-trivial memories. Appendix A.1 gives the derivation.

4.3 Learning rules

Learning combines three local terms. Each acts on a synapse using only the activity of its two neurons.

Hebbian term. Co-active neurons strengthen their connection (Hebb 1949), which stores patterns as attractors:

$$\Delta W_{ij}^{\text{hebb}} = \eta x_i x_j \quad \text{i.e.} \quad \Delta W^{\text{hebb}} = \eta x x^T \quad (5)$$

Temporal association. With the convention that W_{ij} maps neuron j onto neuron i and the update $x(t+1) = \varphi(Wx(t))$, the synapse that should be strengthened is the one from a neuron active now onto a neuron active next:

$$\Delta W_{ij}^{\text{temp}} = \lambda x_i(t+1) x_j(t) \quad \text{i.e.} \quad \Delta W^{\text{temp}} = \lambda x(t+1) x(t)^T \quad (6)$$

This is the sequence analogue of Hebb's rule, with post-synaptic activity taken one step after pre-synaptic activity, consistent with the causal branch of spike-timing-dependent plasticity (Markram et al. 1997; Bi & Poo 1998). If weights are instead indexed as row-vector maps ($x(t+1) = \varphi(x(t) W)$), the outer product is transposed. Using the transposed orientation with column-vector dynamics would store the sequence in reverse.

Homeostatic regularization. Unbounded Hebbian growth is prevented by weight decay, a simplified stand-in for synaptic scaling (Turrigiano et al. 1998; Turrigiano & Nelson 2004):

$$\Delta W_{ij}^{\text{reg}} = -\gamma W_{ij} \quad (7)$$

The combined update is

$$\Delta W = \eta x(t) x(t)^T + \lambda x(t+1) x(t)^T - \gamma W \quad (8)$$

In the gated model the first term accumulates into W_A and the second into W_T . Weight decay makes the store a palimpsest: old memories fade as new ones arrive, at a rate set by γ (Parisi 1986). A normalizing alternative to decay is Oja's rule (Oja 1982), which we include as an ablation.

4.4 Sequence encoding

For a sequence $\{x_1, x_2, \dots, x_k\}$ of patterns with entries ± 1 , the transition matrix is the sum of successor-predecessor outer products:

$$W_T = \frac{1}{N} \sum_{i=1}^{k-1} x_{i+1} x_i^T \quad (9)$$

This induces directional transitions: presenting x_j yields

$$W_T x_j = x_{j+1} + \frac{1}{N} \sum_{i \neq j} x_{i+1} (x_i \cdot x_j) \quad (10)$$

The first term is the desired successor; the second is crosstalk from the other transitions, whose magnitude limits capacity (Section 4.7). A closing term $x_1 x_k^T$ makes the sequence cyclic.

4.5 Gated dynamics

To let one matrix serve both holding and advancing, the recurrent drive is split into attractor and gated transition parts:

$$x(t+1) = \varphi(W_A x(t) + g(t) W_T x(t) + U u(t) + b) \quad (11)$$

where $W_A = (1/N) \sum_{\mu} \xi^{\mu} \xi^{\mu T}$ is the symmetric Hebbian matrix and the gate is a periodic, theta-like signal:

$$g(t) = g_{\max} \cdot \sigma(\kappa [\cos(2\pi t / T) - \theta_g]) \quad (12)$$

with logistic function σ , sharpness κ , period T and threshold $\theta_g \in (-1, 1)$. Two conditions define a working regime. When $g \approx 0$ the network is a symmetric attractor network and holds its state. When $g \approx g_{\max}$ the transition term must outweigh the attractor term, which requires $g_{\max} > 1$ for unit-normalized patterns, with margin for crosstalk. The dwell time in each memory is then set by the period T , and the fraction of each cycle during which the gate is open is set by θ_g . A self-paced alternative, in which a slow adaptation variable opens the gate after a state has been held for a characteristic time, is a natural extension and is not analyzed here.

4.6 Dendritic extension

Replacing the point-neuron sum by K nonlinear branch subunits per neuron gives

$$x_i(t+1) = \varphi(b_i + (U u)_i + \sum_{k=1}^K \sigma_d(\sum_{j \in B_{ik}} W_{ijk} x_j(t) - \theta_{ik})) \quad (13)$$

where B_{ik} is the set of presynaptic neurons contacting branch k of neuron i , σ_d is a sigmoidal branch nonlinearity, and θ_{ik} is a branch threshold. With $K = 1$ and linear σ_d this reduces to Eq. (1). Learning rules (5)–(7) apply per branch, using presynaptic activity and the branch's own output.

4.7 Capacity analysis

Patterns. For dense, uncorrelated binary patterns stored with the Hebbian rule, the maximum number of retrievable patterns scales linearly with network size:

$$P_{\max} \approx \alpha_c N, \quad \alpha_c \approx 0.14 \quad (14)$$

The value $\alpha_c \approx 0.138$ was derived by Amit et al. (1985) and analyzed further by Amit et al. (1987). It is specific to this rule and pattern class. Optimal weights can reach $\alpha = 2$ (Gardner 1988), the projection (pseudo-inverse) rule stores up to about N linearly independent patterns (Personnaz et al. 1986; Kanter & Sompolinsky 1987), and sparse coding at activity level f raises capacity roughly as $1/(f |\ln f|)$ (Willshaw et al. 1969; Tsodyks & Feigel'man 1988).

Sequences. For error-free retrieval of stored patterns, capacity scales as $N/\ln N$ rather than N : McEliece et al. (1987) showed that for the Hopfield model the limit is $N/(4 \ln N)$ if every stored pattern must be a fixed point and $N/(2 \ln N)$ if most must be. Because the crosstalk in Eq. (10) has the same statistics as in hetero-associative recall of independent random patterns, we adopt

$$S_{\max} \approx \frac{N}{c \ln N}, \quad c \in [2, 4] \quad (15)$$

as the working estimate for the number of transitions that can be stored with error-free step-by-step retrieval, and treat it as a hypothesis to be verified in simulation (Section 6, H3). Appendix A.2 sketches the argument.

4.8 Energy model

In a spike-based approximation the energy of a retrieval episode is the number of spikes times the cost per spike:

$$E \approx \sum_t^{T_r} \sum_{i=1}^N s_i(t) E_{\text{spike}} \quad (16)$$

where T_r is the retrieval duration. A more faithful estimate for hardware adds a synaptic-operation cost proportional to fan-out, $E \approx \sum_t \sum_i s_i(t) (E_{\text{spike}} + K_i E_{\text{syn}})$, with K_i the fan-out of neuron i . For rate networks, $s_i(t)$ is estimated from activity, for example $s_i = (1 + x_i)/2$, the firing probability of a unit with activity $x_i \in [-1, 1]$. Efficiency is then posed as a constrained optimization over design parameters θ (size, gain, sparsity, gate period):

$$\min_{\theta} E(\theta) \quad \text{s.t.} \quad \text{Recall accuracy}(\theta) > \theta_{\text{acc}} \quad (17)$$

and reported as an accuracy–energy Pareto front rather than a single number, because the energy per event depends on hardware (Mead 1990; Davies et al. 2018; Merolla et al. 2014).

5 Methods and simulation framework

This section is an implementation plan detailed enough to reproduce the study.

5.1 Environment

Python 3.10 or later with PyTorch (Paszke et al. 2019), NumPy (Harris et al. 2020) and Matplotlib (Hunter 2007). Brian 2 (Stimberg et al. 2019) is optional and used for the spiking extension. Textbook references for the underlying models are (Dayan & Abbott 2001; Gerstner et al. 2014).

5.2 Network architecture and parameters

The pipeline is Input $\rightarrow$ Encoding layer $\rightarrow$ Recurrent core $\rightarrow$ Readout (Figure 2). Table 3 lists the parameters. Values are starting ranges to be tuned on a held-out set of random seeds and then fixed before final experiments.

Table 3. Architecture and parameters (initial ranges; to be tuned and then fixed).

Parameter	Meaning	Initial value or range
N	Recurrent neurons	500, 1 000, 2 000, 5 000
M	Input dimension	50–200
Pattern statistics	Stored items	Dense random ± 1 ($f = 0.5$); sparse ($f \in \{0.05, 0.1\}$); correlated (pairwise overlap 0.1, 0.3)
P, k	Number of patterns; sequence length	Load $\alpha = P/N$ from 0.02 to 0.3; k up to $2N/\ln N$
β	Activation gain	1.5–4, tuned so stored states saturate
η, λ	Hebbian and temporal rates	$\eta = \lambda = 1/N$ (matches Eqs. 9 and 11)
γ	Homeostatic decay per presentation	10^{-5} to 10^{-3}
$g_{\max}, T, \theta_g, \kappa$	Gate amplitude, period, threshold, sharpness	$g_{\max} \in [1.2, 2]$; $T \in 5\text{--}20$ steps; $\theta_g \in [-0.5, 0.5]$; $\kappa = 8$
W_{init}	Initial weights	$N(0, 0.01^2)$
Cue noise	Corruption of the initial state	Bit-flip fraction 0.05–0.4, or Gaussian, matching Task A
Recall horizon	Maximum steps per recall	100
Seeds	Independent replicates	20 per condition

5.3 Training procedure

Procedure 1. Three-phase learning

- Phase 1, pattern learning.** Generate random (or structured) patterns; present each repeatedly; apply the Hebbian update, Eq. (5), to W_A .
- Phase 2, sequence learning.** Feed ordered patterns; apply the temporal update, Eq. (6), to W_T .
- Phase 3, stabilization.** Apply homeostatic regularization, Eq. (7); normalize row norms of W_A and W_T ; verify Eq. (4) at every stored pattern and check that $\rho(\beta W) > 1$ at the origin (Section 4.2).

5.4 Evaluation tasks and metrics

Table 4. Evaluation tasks, protocols and metrics. The recall criterion m^* is fixed before experiments (default 0.95).

Task	Protocol	Metrics and success criteria
A. Pattern recall	Corrupt a stored pattern, $\tilde{x} = x + \varepsilon$; iterate to convergence or the recall horizon	Final overlap m_f ; <i>recall accuracy</i> = fraction of trials with $m_f \geq m^*$; basin radius = largest noise with accuracy ≥ 0.95 ; fraction of spurious end states
B. Sequence recall	Present a prefix of length 1–3; free-run the gated dynamics	Transition accuracy per step; exact full-sequence match rate; timing error in steps
C. Interference	Store P_0 items, then add new memories in stages; test all items after each stage	Retention curve (accuracy vs. items added); forgetting index; distinction between graceful decay and abrupt collapse
D. Scaling	Sweep N ; find the largest load with accuracy ≥ 0.95	$P_{\max}(N)$, $S_{\max}(N)$; scaling exponent from log–log regression with 95% CI
E. Efficiency	Count spikes and synaptic operations per recall (Eq. 16)	Energy proxy per recall; accuracy–energy Pareto front; latency in steps to convergence
F. Hardware robustness	Quantize weights to 2–8 bits; add multiplicative weight noise	Accuracy vs. bit depth and noise level

Pattern recall (A) is measured with an overlap criterion rather than exact equality so that graded-activity states count as recalled. Stability diagnostics accompany every run: the empirical spectral radius of Eq. (3) at stored states and a largest-Lyapunov-exponent estimate of free-running trajectories.

5.5 Baselines

Table 5. Baselines. All are compared at matched parameter count and reported with the same energy proxy.

Baseline	Configuration	Used for
Hopfield (Hebbian and projection rules)	Symmetric weights; asynchronous sign updates (Hopfield 1982; Personnaz et al. 1986)	Tasks A, C, D
Modern Hopfield	Dense associative memory (Krotov & Hopfield 2016; Ramsauer et al. 2021)	Upper reference for pattern capacity
Echo state network	Fixed random reservoir, spectral radius near 0.9, ridge-regression readout (Jaeger 2001; Lukoševičius & Jaeger 2009)	Tasks B, D
LSTM	One layer, hidden size set for equal parameter count, trained with BPTT (Hochreiter & Schmidhuber 1997)	Tasks B, C, E

Metrics reported for every method are accuracy, energy proxy, latency and capacity. Each condition is run with 20 seeds; results are reported as mean with a bootstrap 95% confidence interval, and hyperparameters of the baselines are tuned with the same budget as for the proposed model. Code, seeds and configuration files are to be released with the paper.

5.6 Reference implementation

The following minimal PyTorch listing implements Eqs. (1), (5)–(7) and the successor–predecessor convention of Eq. (6). It is a starting point for the full experiments; the gate of Eq. (12) and the dendritic extension are added on top of it.

```

import torch

torch.manual_seed(0)
N, BETA = 1000, 2.0                                # neurons, activation gain
W = torch.randn(N, N) * 0.01                        # W[i, j] : synapse from neuron j onto neuron i

def step(x, W, u=None, b=0.0):
    """One update, Eq. (1):  $x(t+1) = \tanh(\beta * (W x + u + b))$ .
    u is the already-projected input U @ u(t)."""
    a = W @ x + b
    if u is not None:
        a = a + u
    return torch.tanh(BETA * a)

def hebbian(W, x, lr=1.0 / N):
    """Auto-associative term, Eq. (5): stores x as an attractor (symmetric)."""
    return W + lr * torch.outer(x, x)

def temporal(W, x_now, x_next, lr=1.0 / N):
    """Hetero-associative term, Eq. (6): drives  $x_{\text{now}} \rightarrow x_{\text{next}}$  under step().
    outer(post, pre) = outer( $x_{\text{next}}$ ,  $x_{\text{now}}$ )."""
    return W + lr * torch.outer(x_next, x_now)

def decay(W, gamma=1e-4):
    """Homeostatic regularization, Eq. (7)."""
    return W * (1.0 - gamma)

def recall(W, x0, steps=100):
    """Free-running retrieval; returns the trajectory as a (steps+1, N) tensor."""
    traj, x = [x0], x0
    for _ in range(steps):
        x = step(x, W)
        traj.append(x)
    return torch.stack(traj)

```

6 Results: targets and protocol

The proposed simulations are expected to demonstrate the five outcomes listed below (robust recall, stable convergence, scalable capacity, graceful forgetting and energy efficiency). Because the experiments of Section 5 have not been run, Table 6 states each expected outcome as a falsifiable hypothesis, with the task that will test it and the quantity that will be reported. This section is to be replaced with measured results, figures and confidence intervals once the study is complete.

Table 6. Pre-specified target outcomes.

ID	Outcome	Hypothesis to be tested	Task and metric	Status
H1	Noise-robust recall	Recall accuracy of at least 95% under cue noise. The noise level and load at which this holds are to be fixed before the experiments.	A; recall accuracy, basin radius	pending
H2	Stable convergence	Trajectories converge to stored states within the recall horizon; the empirical spectral radius of Eq. (3) is below 1 at every stored state; free-running dynamics have a non-positive largest Lyapunov exponent between gate openings.	A, B; convergence steps, $\rho(J)$, Lyapunov exponent	pending
H3	Scalable capacity	Pattern capacity grows linearly with N over the tested range (Eq. 14); error-free sequence capacity follows $N/(c \ln N)$ (Eq. 15).	D; $P_{\max}(N)$, $S_{\max}(N)$, exponents	pending
H4	Graceful forgetting under interference	With $\gamma > 0$, retention declines gradually as new memories are added (palimpsest behaviour) instead of collapsing abruptly at capacity, in contrast to unregularized Hopfield storage.	C; retention curve, forgetting index	pending
H5	Energy efficiency	At matched accuracy, the event-based energy proxy of the model is lower than that of the LSTM and the dense echo state network. The size of the gap is a result, not an assumption.	E; energy proxy, Pareto front	pending

Resistance to catastrophic forgetting is stated as H4 rather than as a result. Catastrophic forgetting is the abrupt loss of old items when new ones are learned (French 1999). Hopfield-type storage above capacity does this, whereas bounded or decaying weights degrade gradually (Parisi 1986; Fusi et al. 2005). The correct measure is therefore the shape of the retention curve, not a single accuracy value. Methods that address catastrophic forgetting in gradient-trained networks (Kirkpatrick et al. 2017) are not directly comparable, because learning here is local and one-shot.

7 Discussion

A unified architecture. If confirmed, the results would support a single recurrent substrate in which items, order, and control coexist without a separate memory module. The symmetric and asymmetric components of the recurrent matrix have separable roles (holding versus advancing), and a single gate variable switches between them. This is testable: the model predicts that dwell time per item scales with the gate period T and that removing the gate freezes retrieval in the first attractor.

Relation to biology. Auto-associative attractor retrieval maps onto recurrent-collateral networks in hippocampal CA3, with pattern completion from partial cues and pattern separation upstream (Treves & Rolls 1994; Rolls 2013); rhythmic gating of item order resembles theta–gamma coding (Lisman & Jensen 2013); and gating by prefrontal circuits corresponds to models of controlled working memory (Miller & Cohen 2001; O’Reilly & Frank 2006). Short-term synaptic dynamics offer an alternative, activity-silent route to short-term memory (Mongillo et al. 2008), not modeled here and a natural addition. The model is deliberately abstract: units are point rate neurons, plasticity is one-shot, and no claim is made about specific anatomy.

Implications for neuromorphic engineering. Weights W that are both memory and compute map directly onto in-memory computing with crossbar arrays (Indiveri & Liu 2015; Sebastian et al. 2020) and onto event-driven neuromorphic processors (Davies et al. 2018; Merolla et al. 2014). Local learning rules of the form in Eqs. (5)–(7) need only pre- and post-synaptic activity, which suits on-chip learning.

Relation to modern sequence models. Structured state-space models (Gu et al. 2022; Gu & Dao 2023) also carry information in a recurrent state, but their memory is a learned linear system trained by gradient descent. The attractor view here differs in that memory items are explicit stable states that can be completed from partial cues, at the cost of the capacity bounds of Section 4.7.

8 Limitations

Training stability. Unbounded Hebbian growth, spurious attractors, and chaotic regimes at high gain (Sompolinsky et al. 1988) can all destroy retrieval. Planned mitigations are weight decay (Eq. 7), normalization by Oja’s rule (Oja 1982), gain control, and continuous monitoring of the Jacobian spectrum and Lyapunov exponent. The gated asymmetric dynamics lack a Lyapunov function, so convergence is established empirically, not proved.

Interference effects. Crosstalk between patterns grows with load and with pattern correlation, and capacity degrades for non-random data. Options include the projection rule (Personnaz et al. 1986; Kanter & Sompolinsky 1987), sparse coding (Willshaw et al. 1969; Tsodyks & Feigel’man 1988), pattern separation before storage (Rolls 2013), and cascade synapses that retain older memories longer (Fusi et al. 2005), as well as complementary fast and slow stores (McClelland et al. 1995).

Hardware precision limits. Analog memory devices have finite resolution, variability and drift. Task F quantifies sensitivity to weight precision and noise; the memristive implementations of Section 9 face these constraints directly (Strukov et al. 2008; Prezioso et al. 2015).

Further limitations. Capacity results are derived for random patterns and may not carry over to natural data; the scaling claims will hold only for the sizes tested; and the model uses idealized point neurons in its base form. The energy model counts events, not device-level power.

9 Future work

- **Spiking implementations.** Port the model to spiking neurons with spike-timing-dependent plasticity, trained locally or with surrogate gradients (Neftci et al. 2019; Bellec et al. 2020), and simulate in Brian 2 (Stimberg et al. 2019). This also gives a direct measurement of Eq. (16).
- **Memristive hardware.** Map W onto crossbar arrays of memristive devices (Strukov et al. 2008; Prezioso et al. 2015) and study the effect of device nonidealities on capacity, following the in-memory computing literature (Sebastian et al. 2020; Indiveri & Liu 2015).
- **Online learning.** Replace batch presentation by continual, one-shot updates with bounded synapses, so that memories are acquired and forgotten gracefully (Fusi et al. 2005; Parisi 1986).
- **Multimodal integration.** Bind modality-specific attractor sets through hetero-associative links, so that a cue in one modality completes the corresponding state in another.
- **Theory.** Mean-field analysis of gated asymmetric dynamics, and capacity of the dendritic extension in Eq. (13) (Poirazi & Mel 2001).

10 Conclusion

Memory embedded in neural dynamics provides a scalable and biologically grounded alternative to modular storage systems. We formalized memories as stable attractors and sequences as gated trajectories through them within one recurrent network, gave conditions for stability, capacity estimates, and an energy model, and specified the simulation study needed to test each claim. The framework connects attractor theory, temporal plasticity, dendritic computation and neuromorphic engineering, and it makes specific, falsifiable predictions about capacity scaling, dwell time and forgetting.

Data and code availability. Simulation code and configurations will be released with the final version.

References

- Amari, S. (1977). Dynamics of pattern formation in lateral-inhibition type neural fields. *Biological Cybernetics*, 27(2), 77–87.
<https://doi.org/10.1007/BF00337259>
- Amit, D. J., Gutfreund, H., & Sompolinsky, H. (1985). Storing infinite numbers of patterns in a spin-glass model of neural networks. *Physical Review Letters*, 55(14), 1530–1533.
<https://doi.org/10.1103/PhysRevLett.55.1530>
- Amit, D. J., Gutfreund, H., & Sompolinsky, H. (1987). Statistical mechanics of neural networks near saturation. *Annals of Physics*, 173(1), 30–67.
[https://doi.org/10.1016/0003-4916\(87\)90092-3](https://doi.org/10.1016/0003-4916(87)90092-3)
- Amit, D. J. (1988). Neural networks counting chimes. *Proceedings of the National Academy of Sciences*, 85(7), 2141–2145.
<https://doi.org/10.1073/pnas.85.7.2141>
- Attwell, D., & Laughlin, S. B. (2001). An energy budget for signaling in the grey matter of the brain. *Journal of Cerebral Blood Flow & Metabolism*, 21(10), 1133–1145.
<https://doi.org/10.1097/00004647-200110000-00001>
- Backus, J. (1978). Can programming be liberated from the von Neumann style? A functional style and its algebra of programs. *Communications of the ACM*, 21(8), 613–641.
<https://doi.org/10.1145/359576.359579>
- Bellec, G., Scherr, F., Subramoney, A., Hajek, E., Salaj, D., Legenstein, R., & Maass, W. (2020). A solution to the learning dilemma for recurrent networks of spiking neurons. *Nature Communications*, 11, 3625.
<https://doi.org/10.1038/s41467-020-17236-y>
- Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2), 157–166.
<https://doi.org/10.1109/72.279181>
- Ben-Yishai, R., Bar-Or, R. L., & Sompolinsky, H. (1995). Theory of orientation tuning in visual cortex. *Proceedings of the National Academy of Sciences*, 92(9), 3844–3848.
<https://doi.org/10.1073/pnas.92.9.3844>
- Bertschinger, N., & Natschläger, T. (2004). Real-time computation at the edge of chaos in recurrent neural networks. *Neural Computation*, 16(7), 1413–1436.
<https://doi.org/10.1162/089976604323057443>
- Bi, G.-q., & Poo, M.-m. (1998). Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and postsynaptic cell type. *Journal of Neuroscience*, 18(24), 10464–10472.
<https://doi.org/10.1523/JNEUROSCI.18-24-10464.1998>
- Buzsáki, G. (2010). Neural syntax: cell assemblies, synapsembles, and readers. *Neuron*, 68(3), 362–385.
<https://doi.org/10.1016/j.neuron.2010.09.023>
- Chaudhuri, R., & Fiete, I. (2016). Computational principles of memory. *Nature Neuroscience*, 19(3), 394–403.
<https://doi.org/10.1038/nn.4237>
- Davies, M., Srinivasa, N., Lin, T.-H., Chinya, G., Cao, Y., Choday, S. H., et al. (2018). Loihi: a neuromorphic manycore processor with on-chip learning. *IEEE Micro*, 38(1), 82–99.
<https://doi.org/10.1109/MM.2018.112130359>
- Dayan, P., & Abbott, L. F. (2001). *Theoretical Neuroscience: Computational and Mathematical Modeling of Neural Systems*. MIT Press.
- Demircigil, M., Heusel, J., Löwe, M., Uppang, S., & Vermet, F. (2017). On a model of associative memory with huge storage capacity. *Journal of Statistical Physics*, 168(2), 288–299.
<https://doi.org/10.1007/s10955-017-1806-y>
- French, R. M. (1999). Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4), 128–135.
[https://doi.org/10.1016/S1364-6613\(99\)01294-2](https://doi.org/10.1016/S1364-6613(99)01294-2)
- Friston, K. (2010). The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2), 127–138.
<https://doi.org/10.1038/nrn2787>
- Fusi, S., Drew, P. J., & Abbott, L. F. (2005). Cascade models of synaptically stored memories. *Neuron*, 45(4), 599–611.
<https://doi.org/10.1016/j.neuron.2005.02.001>

- Ganguli, S., Huh, D., & Sompolinsky, H. (2008). Memory traces in dynamical systems. *Proceedings of the National Academy of Sciences*, 105(48), 18970–18975.
<https://doi.org/10.1073/pnas.0804451105>
- Gardner, E. (1988). The space of interactions in neural network models. *Journal of Physics A: Mathematical and General*, 21(1), 257–270.
<https://doi.org/10.1088/0305-4470/21/1/030>
- Gerstner, W., Kistler, W. M., Naud, R., & Paninski, L. (2014). *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press.
<https://neurondynamics.epfl.ch/>
- Goldman-Rakic, P. S. (1995). Cellular basis of working memory. *Neuron*, 14(3), 477–485.
[https://doi.org/10.1016/0896-6273\(95\)90304-6](https://doi.org/10.1016/0896-6273(95)90304-6)
- Graves, A., Wayne, G., & Danihelka, I. (2014). Neural Turing machines. *arXiv preprint arXiv:1410.5401*.
<https://arxiv.org/abs/1410.5401>
- Gu, A., Goel, K., & Ré, C. (2022). Efficiently modeling long sequences with structured state spaces. *International Conference on Learning Representations (ICLR)*. arXiv:2111.00396.
<https://arxiv.org/abs/2111.00396>
- Gu, A., & Dao, T. (2023). Mamba: linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.
<https://arxiv.org/abs/2312.00752>
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.
<https://doi.org/10.1038/s41586-020-2649-2>
- Hebb, D. O. (1949). *The Organization of Behavior: A Neuropsychological Theory*. Wiley.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
<https://doi.org/10.1162/neco.1997.9.8.1735>
- Hopfield, J. J. (1982). Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences*, 79(8), 2554–2558.
<https://doi.org/10.1073/pnas.79.8.2554>
- Hopfield, J. J. (1984). Neurons with graded response have collective computational properties like those of two-state neurons. *Proceedings of the National Academy of Sciences*, 81(10), 3088–3092.
<https://doi.org/10.1073/pnas.81.10.3088>
- Horowitz, M. (2014). Computing’s energy problem (and what we can do about it). *2014 IEEE International Solid-State Circuits Conference (ISSCC) Digest of Technical Papers*, 10–14.
<https://doi.org/10.1109/ISSCC.2014.6757323>
- Hunter, J. D. (2007). Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
<https://doi.org/10.1109/MCSE.2007.55>
- Indiveri, G., & Liu, S.-C. (2015). Memory and information processing in neuromorphic systems. *Proceedings of the IEEE*, 103(8), 1379–1397.
<https://doi.org/10.1109/JPROC.2015.2444094>
- Jaeger, H. (2001). The “echo state” approach to analysing and training recurrent neural networks. *GMD Technical Report 148*, German National Research Center for Information Technology.
<https://www.ai.rug.nl/minds/uploads/EchoStatesTechRep.pdf>
- Jaeger, H., & Haas, H. (2004). Harnessing nonlinearity: predicting chaotic systems and saving energy in wireless communication. *Science*, 304(5667), 78–80.
<https://doi.org/10.1126/science.1091277>
- Kanter, I., & Sompolinsky, H. (1987). Associative recall of memory without errors. *Physical Review A*, 35(1), 380–392.
<https://doi.org/10.1103/PhysRevA.35.380>
- Khona, M., & Fiete, I. R. (2022). Attractor and integrator networks in the brain. *Nature Reviews Neuroscience*, 23(12), 744–766.
<https://doi.org/10.1038/s41583-022-00642-0>
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., et al. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521–3526.
<https://doi.org/10.1073/pnas.1611835114>

- Kleinfeld, D. (1986). Sequential state generation by model neural networks. *Proceedings of the National Academy of Sciences*, 83(24), 9469–9473.
<https://doi.org/10.1073/pnas.83.24.9469>
- Krotov, D., & Hopfield, J. J. (2016). Dense associative memory for pattern recognition. *Advances in Neural Information Processing Systems*, 29. arXiv:1606.01164.
<https://arxiv.org/abs/1606.01164>
- Larkum, M. E., Zhu, J. J., & Sakmann, B. (1999). A new cellular mechanism for coupling inputs arriving at different cortical layers. *Nature*, 398(6725), 338–341.
<https://doi.org/10.1038/18686>
- Lennie, P. (2003). The cost of cortical computation. *Current Biology*, 13(6), 493–497.
[https://doi.org/10.1016/S0960-9822\(03\)00135-0](https://doi.org/10.1016/S0960-9822(03)00135-0)
- Lisman, J. E., & Jensen, O. (2013). The theta-gamma neural code. *Neuron*, 77(6), 1002–1016.
<https://doi.org/10.1016/j.neuron.2013.03.007>
- London, M., & Häusser, M. (2005). Dendritic computation. *Annual Review of Neuroscience*, 28, 503–532.
<https://doi.org/10.1146/annurev.neuro.28.061604.135703>
- Lukoševičius, M., & Jaeger, H. (2009). Reservoir computing approaches to recurrent neural network training. *Computer Science Review*, 3(3), 127–149.
<https://doi.org/10.1016/j.cosrev.2009.03.005>
- Maass, W. (1997). Networks of spiking neurons: the third generation of neural network models. *Neural Networks*, 10(9), 1659–1671.
[https://doi.org/10.1016/S0893-6080\(97\)00011-7](https://doi.org/10.1016/S0893-6080(97)00011-7)
- Maass, W., Natschläger, T., & Markram, H. (2002). Real-time computing without stable states: a new framework for neural computation based on perturbations. *Neural Computation*, 14(11), 2531–2560.
<https://doi.org/10.1162/089976602760407955>
- Mante, V., Sussillo, D., Shenoy, K. V., & Newsome, W. T. (2013). Context-dependent computation by recurrent dynamics in prefrontal cortex. *Nature*, 503(7474), 78–84.
<https://doi.org/10.1038/nature12742>
- Markram, H., Lübke, J., Frotscher, M., & Sakmann, B. (1997). Regulation of synaptic efficacy by coincidence of postsynaptic APs and EPSPs. *Science*, 275(5297), 213–215.
<https://doi.org/10.1126/science.275.5297.213>
- McClelland, J. L., McNaughton, B. L., & O'Reilly, R. C. (1995). Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3), 419–457.
<https://doi.org/10.1037/0033-295X.102.3.419>
- McEliece, R. J., Posner, E. C., Rodemich, E. R., & Venkatesh, S. S. (1987). The capacity of the Hopfield associative memory. *IEEE Transactions on Information Theory*, 33(4), 461–482.
<https://doi.org/10.1109/TIT.1987.1057328>
- Mead, C. (1990). Neuromorphic electronic systems. *Proceedings of the IEEE*, 78(10), 1629–1636.
<https://doi.org/10.1109/5.58356>
- Merolla, P. A., Arthur, J. V., Alvarez-Icaza, R., Cassidy, A. S., Sawada, J., Akopyan, F., et al. (2014). A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, 345(6197), 668–673.
<https://doi.org/10.1126/science.1254642>
- Miller, E. K., & Cohen, J. D. (2001). An integrative theory of prefrontal cortex function. *Annual Review of Neuroscience*, 24, 167–202.
<https://doi.org/10.1146/annurev.neuro.24.1.167>
- Mongillo, G., Barak, O., & Tsodyks, M. (2008). Synaptic theory of working memory. *Science*, 319(5869), 1543–1546.
<https://doi.org/10.1126/science.1150769>
- Neftci, E. O., Mostafa, H., & Zenke, F. (2019). Surrogate gradient learning in spiking neural networks: bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6), 51–63.
<https://doi.org/10.1109/MSP.2019.2931595>
- Oja, E. (1982). Simplified neuron model as a principal component analyzer. *Journal of Mathematical Biology*, 15(3), 267–273.
<https://doi.org/10.1007/BF00275687>

- O'Reilly, R. C., & Frank, M. J. (2006). Making working memory work: a computational model of learning in the prefrontal cortex and basal ganglia. *Neural Computation*, 18(2), 283–328.
<https://doi.org/10.1162/089976606775093909>
- Parisi, G. (1986). A memory which forgets. *Journal of Physics A: Mathematical and General*, 19(10), L617–L620.
<https://doi.org/10.1088/0305-4470/19/10/011>
- Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the difficulty of training recurrent neural networks. *Proceedings of the 30th International Conference on Machine Learning (ICML)*. arXiv:1211.5063.
<https://arxiv.org/abs/1211.5063>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). PyTorch: an imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32. arXiv:1912.01703.
<https://arxiv.org/abs/1912.01703>
- Personnaz, L., Guyon, I., & Dreyfus, G. (1986). Collective computational properties of neural networks: new learning mechanisms. *Physical Review A*, 34(5), 4217–4228.
<https://doi.org/10.1103/PhysRevA.34.4217>
- Poirazi, P., & Mel, B. W. (2001). Impact of active dendrites and structural plasticity on the memory capacity of neural tissue. *Neuron*, 29(3), 779–796.
[https://doi.org/10.1016/S0896-6273\(01\)00252-5](https://doi.org/10.1016/S0896-6273(01)00252-5)
- Poirazi, P., Brannon, T., & Mel, B. W. (2003). Pyramidal neuron as two-layer neural network. *Neuron*, 37(6), 989–999.
[https://doi.org/10.1016/S0896-6273\(03\)00149-1](https://doi.org/10.1016/S0896-6273(03)00149-1)
- Prezioso, M., Merrih-Bayat, F., Hoskins, B. D., Adam, G. C., Likharev, K. K., & Strukov, D. B. (2015). Training and operation of an integrated neuromorphic network based on metal-oxide memristors. *Nature*, 521(7550), 61–64.
<https://doi.org/10.1038/nature14441>
- Rabinovich, M., Volkovskii, A., Lecanda, P., Huerta, R., Abarbanel, H. D. I., & Laurent, G. (2001). Dynamical encoding by networks of competing neuron groups: winnerless competition. *Physical Review Letters*, 87(6), 068102.
<https://doi.org/10.1103/PhysRevLett.87.068102>
- Rabinovich, M. I., Huerta, R., & Laurent, G. (2008). Transient dynamics for neural processing. *Science*, 321(5885), 48–50.
<https://doi.org/10.1126/science.1155564>
- Rajan, K., Abbott, L. F., & Sompolinsky, H. (2010). Stimulus-dependent suppression of chaos in recurrent neural networks. *Physical Review E*, 82(1), 011903.
<https://doi.org/10.1103/PhysRevE.82.011903>
- Ramsauer, H., Schäfl, B., Lehner, J., Seidl, P., Widrich, M., Adler, T., et al. (2021). Hopfield networks is all you need. *International Conference on Learning Representations (ICLR)*. arXiv:2008.02217.
<https://arxiv.org/abs/2008.02217>
- Rao, R. P. N., & Ballard, D. H. (1999). Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1), 79–87.
<https://doi.org/10.1038/4580>
- Rolls, E. T. (2013). The mechanisms for pattern completion and pattern separation in the hippocampus. *Frontiers in Systems Neuroscience*, 7, 74.
<https://doi.org/10.3389/fnsys.2013.00074>
- Sebastian, A., Le Gallo, M., Khaddam-Aljameh, R., & Eleftheriou, E. (2020). Memory devices and applications for in-memory computing. *Nature Nanotechnology*, 15(7), 529–544.
<https://doi.org/10.1038/s41565-020-0655-z>
- Seung, H. S. (1996). How the brain keeps the eyes still. *Proceedings of the National Academy of Sciences*, 93(23), 13339–13344.
<https://doi.org/10.1073/pnas.93.23.13339>
- Skaggs, W. E., & McNaughton, B. L. (1996). Replay of neuronal firing sequences in rat hippocampus during sleep following spatial experience. *Science*, 271(5257), 1870–1873.
<https://doi.org/10.1126/science.271.5257.1870>
- Sompolinsky, H., & Kanter, I. (1986). Temporal association in asymmetric neural networks. *Physical Review Letters*, 57(22), 2861–2864.
<https://doi.org/10.1103/PhysRevLett.57.2861>

- Sompolinsky, H., Crisanti, A., & Sommers, H. J. (1988). Chaos in random neural networks. *Physical Review Letters*, 61(3), 259–262.
<https://doi.org/10.1103/PhysRevLett.61.259>
- Song, S., Miller, K. D., & Abbott, L. F. (2000). Competitive Hebbian learning through spike-timing-dependent synaptic plasticity. *Nature Neuroscience*, 3(9), 919–926.
<https://doi.org/10.1038/78829>
- Stimberg, M., Brette, R., & Goodman, D. F. M. (2019). Brian 2, an intuitive and efficient neural simulator. *eLife*, 8, e47314.
<https://doi.org/10.7554/eLife.47314>
- Strukov, D. B., Snider, G. S., Stewart, D. R., & Williams, R. S. (2008). The missing memristor found. *Nature*, 453(7191), 80–83.
<https://doi.org/10.1038/nature06932>
- Sussillo, D., & Abbott, L. F. (2009). Generating coherent patterns of activity from chaotic neural networks. *Neuron*, 63(4), 544–557.
<https://doi.org/10.1016/j.neuron.2009.07.018>
- Treves, A., & Rolls, E. T. (1994). Computational analysis of the role of the hippocampus in memory. *Hippocampus*, 4(3), 374–391.
<https://doi.org/10.1002/hipo.450040319>
- Tsodyks, M. V., & Feigel'man, M. V. (1988). The enhanced storage capacity in neural networks with low activity level. *Europhysics Letters*, 6(2), 101–105.
<https://doi.org/10.1209/0295-5075/6/2/002>
- Turrigiano, G. G., Leslie, K. R., Desai, N. S., Rutherford, L. C., & Nelson, S. B. (1998). Activity-dependent scaling of quantal amplitude in neocortical neurons. *Nature*, 391(6670), 892–896.
<https://doi.org/10.1038/36103>
- Turrigiano, G. G., & Nelson, S. B. (2004). Homeostatic plasticity in the developing nervous system. *Nature Reviews Neuroscience*, 5(2), 97–107.
<https://doi.org/10.1038/nrn1327>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30. arXiv:1706.03762.
<https://arxiv.org/abs/1706.03762>
- Vyas, S., Golub, M. D., Sussillo, D., & Shenoy, K. V. (2020). Computation through neural population dynamics. *Annual Review of Neuroscience*, 43, 249–275.
<https://doi.org/10.1146/annurev-neuro-092619-094115>
- Wang, X.-J. (2001). Synaptic reverberation underlying mnemonic persistent activity. *Trends in Neurosciences*, 24(8), 455–463.
[https://doi.org/10.1016/S0166-2236\(00\)01868-3](https://doi.org/10.1016/S0166-2236(00)01868-3)
- Willshaw, D. J., Buneman, O. P., & Longuet-Higgins, H. C. (1969). Non-holographic associative memory. *Nature*, 222(5197), 960–962.
<https://doi.org/10.1038/222960a0>

A Derivation notes

A.1 Jacobian and the role of saturation

Write the fixed point of Eq. (1) with $u = 0$ as $x^* = \varphi(a^*)$, $a^* = Wx^* + b$. A small perturbation δ obeys, to first order,

$$\delta(t+1) = \text{diag}(\varphi'(a^*)) W \delta(t) = J \delta(t) \quad (\text{A1})$$

so δ decays if $\rho(J) < 1$. For $\varphi(z) = \tanh(\beta z)$, $\varphi'(a) = \beta(1 - \tanh^2(\beta a))$, which is small for saturated units. At the origin $\varphi'(0) = \beta$ and $J = \beta W$. For a single stored pattern with $W = \xi \xi^T / N$ and $\xi \in \{\pm 1\}^N$, W has one non-zero eigenvalue equal to 1, so the origin is unstable exactly when $\beta > 1$, and the stable fixed point satisfies $m = \tanh(\beta m)$ for the overlap m . For $\beta > 1$ the non-zero solution exists and has $\rho(J) = \beta(1 - m^2) < 1$.

A.2 Crosstalk in sequence retrieval

Let $x_1, \dots, x_k$ be independent random vectors with entries ± 1 and W_T as in Eq. (9). For neuron n and the current state x_j , Eq. (10) gives

$$(W_T x_j)_n = x_{j+1,n} + C_n, \quad C_n = \frac{1}{N} \sum_{i \neq j} x_{i+1,n} (x_i \cdot x_j) \quad (\text{A2})$$

For large N the crosstalk C_n is approximately Gaussian with zero mean and variance $(k-2)/N \approx k/N$. Neuron n is read out incorrectly when $|C_n|$ exceeds 1 with the wrong sign, which happens with probability about $Q(\sqrt{N/k}) \approx \exp(-N/2k)$. For a typical step to be error-free across all N neurons we need $N \exp(-N/2k) \rightarrow 0$, that is $N/(2k) > \ln N$, so $k \leq N/(2 \ln N)$. If every step of every neuron must be correct at once, the union runs over kN events and $N/(2k)$ must exceed $\ln(kN) \approx 2 \ln N$, so $k \leq N/(4 \ln N)$. These are the two constants ($c = 2$ and $c = 4$) obtained for the Hopfield model by McEliece et al. (1987). This is a heuristic argument for independent patterns; correlated patterns and the effect of the attractor term W_A are not covered.

B Revision notes for the authors

This appendix records how this document differs from the original draft, so that each change can be accepted or reversed. It is intended to be removed before submission.

Table B1. Changes relative to the original draft.

#	Item	Original draft	This document and why
1	Direction of temporal association	$\Delta W_{ij} = \lambda x_i(t)x_j(t+1)$; $W \approx \sum x_i x_{i+1}^T$; code <code>outer(x1, x2)</code>	Successor–predecessor form $x_{i+1}x_i^T$ (Eqs. 6, 9) and <code>outer(x_next, x_now)</code> . With $x(t+1) = \varphi(Wx(t))$, the original form maps x_{i+1} to x_i and would replay sequences backwards.
2	Stability condition	$\rho(W \cdot \varphi') < 1$ (elementwise product); fixed point without bias	Jacobian $\text{diag}(\varphi')W$ (Eq. 3) and bias included in Eq. (2); added the requirement that the origin be unstable for memories to be retrievable.
3	Sequence capacity	$S_{\max} \approx N/\log N$	$N/(c \ln N)$ with $c \in [2, 4]$, attributed to McEliece et al. (1987) and labeled a hypothesis for sequences. The $0.14N$ figure is now stated as specific to Hebbian storage of random binary patterns.
4	Numerical results	“95% recall accuracy”, “stable convergence”, “linear scaling”, “resistance to catastrophic forgetting” stated as findings	Restated as hypotheses H1–H5 with tasks and metrics (Table 6), because the simulations in Section 5 have not been run. Replace with measured data before submission.
5	Forgetting claim	Resistance to catastrophic forgetting	Reframed as graceful (palimpsest) forgetting with decay $\gamma > 0$, which is what the model predicts; catastrophic collapse above capacity is expected without decay.
6	Dendritic computation and gating	Named in the abstract and Section 3.3; no equations	Added Eq. (11)–(12) for gating, Eq. (13) for dendritic subunits, and Figures 1–2. These are new formalizations and should be reviewed by the authors.
7	Energy model	$E \approx \sum \sum s_i E_{\text{spike}}$	Kept as Eq. (16); added a fan-out term and a Pareto-front reporting convention.
8	Reference code	Global weight matrix; no gain; <code>outer(x1, x2)</code>	Pure functions, explicit gain β , corrected outer-product order, recall loop, and $1/N$ scaling.
9	Author details and declarations	None	Single-author, independent-researcher byline with email address; no placeholders remain.
10	References	Family names only	Full citations added with DOI or arXiv links. Verify each entry (volume, pages, DOI) against the publisher record before submission.